

Kesimpulan utama

Analisismu benar untuk arsitektur saat ini: selama viewer menerima teamKey, menjadi anggota folder Drive, dan snapshot masuk ke SQLite, single-use guarantee sudah tidak mungkin dipertahankan.

Tetapi ada desain yang jauh lebih kuat tanpa membangun server sendiri:

> Jangan jadikan single-use viewer sebagai role Team. Jadikan ia sebuah device-bound, at-most-once encrypted view capsule.

Jaminannya harus dirumuskan secara sempit:

> Satu kapsul hanya dapat didekripsi satu kali pada satu perangkat yang didukung, lalu hanya tersedia selama satu sesi foreground dan tidak disimpan sebagai plaintext.

Bukan:

> Viewer dijamin hanya pernah melihat atau mengingat data satu kali.

Jaminan kedua mustahil karena pengguna dapat memotret layar, mengingat data, atau pada perangkat yang sudah dikompromikan menangkap plaintext setelah dekripsi.

1. Pisahkan VIEWER dan ONE_TIME_VIEWER

Jangan modifikasi sedikit alur joinTeam() yang ada. Buat dua konsep berbeda:

| Mode | Semantik |

|---|---|

| TEAM_VIEWER | Anggota persisten, dapat sync berkali-kali, memiliki akses read-only |

| ONE_TIME_VIEWER | Bukan anggota Team, tidak memiliki teamKey, tidak memiliki workspace lokal |

| OWNER/EDITOR | Tetap memakai arsitektur Team sekarang |

Untuk ONE_TIME_VIEWER, hal berikut tidak boleh pernah terjadi:

```
teamKeyStore.store(teamId, teamKey)

repository.joinTeam(...)

INSERT INTO team_workspaces ...

drive.addMember(teamFolderId, viewer)

DriveSyncRuntime.start(...)
```

Sebaiknya ONE_TIME_VIEWER bahkan tidak direpresentasikan sebagai TeamRole. Gunakan tipe terpisah agar tidak ada kode yang tanpa sengaja menganggapnya sebagai anggota Team:

```
sealed interface AccessMode {

    data class TeamMember(val role: TeamRole) : AccessMode

    data object OneTimeCapsule : AccessMode

}
```

2. Ubah unit akses dari snapshot menjadi ViewCapsule

Owner tidak mengirim teamKey. Owner membuat proyeksi data khusus untuk sekali lihat.

Contoh:

```
data class ViewProjection(
    val generatedAt: Instant,
    val period: DateRange,
    val summary: FinancialSummary,
    val transactions: List<ViewTransaction>,
    val budgets: List<ViewBudget>
)
```

Jangan selalu memasukkan seluruh database. Owner sebaiknya menentukan:

- periode transaksi;
- akun yang boleh dilihat;
- apakah nominal detail ditampilkan;
- apakah catatan transaksi disertakan;
- apakah viewer hanya melihat ringkasan;
- apakah data sensitif tertentu disamarkan.

Dengan demikian, kebocoran maksimum dibatasi pada isi kapsul, bukan seluruh Team.

Format kapsul kira-kira:

```
ViewCapsule
|-- manifest
| |-- formatVersion
| |-- capsuleId
| |-- teamId
| |-- ownerKeyId
| |-- targetEmailHash
| |-- targetDeviceKeyId
| |-- issuedAt
| |-- expiresAt
| |-- dataScope
| +-- ciphertextHash
|-- encryptedContentKey
|-- nonce
```

```
|-- encryptedProjection
+-- ownerSignature
```

ownerSignature harus mencakup seluruh manifest, hash ciphertext, nonce, dan wrapped key agar komponen tidak dapat dipindahkan antar-kapsul.

3. Wajib device-bound: invitation satu arah tidak cukup

Invitation sekarang adalah bearer credential:

```
siapa pun yang memperoleh code + secret dapat mencoba menggunakannya
```

targetEmailHash mengikatnya ke akun, tetapi tidak secara kriptografis mengikat ke satu perangkat atau satu instalasi aplikasi. Karena itu, single-use yang kuat membutuhkan handshake dua tahap.

Tahap A — Owner membuat offer

Owner membuat:

```
OneTimeViewOffer
- offerId
- teamId
- targetEmailHash
- requestedScope
- expiry
- ownerSigningPublicKey
- randomChallenge
- ownerSignature
```

Belum ada data finansial atau teamKey di sini.

Tahap B — Viewer membuat device request

Viewer memindai offer, lalu perangkat membuat key pair di Android Keystore:

```
viewerWrappingKey
- private key: non-exportable
- public key: dikirim ke owner
- hardware-backed jika tersedia
- user authentication required
- single-use/limited-use jika didukung
```

Sejak API 31, Android menyediakan `setMaxUsageCount()` serta feature flags untuk mengetahui apakah limited-use atau single-use key ditekankan oleh hardware. Jangan mengklaim jaminan kuat jika perangkat tidak memiliki `FEATURE_KEYSTORE_SINGLE_USE_KEY` atau `FEATURE_KEYSTORE_LIMITED_USE_KEY`.

Viewer kemudian membuat request:

```
OneTimeViewDeviceRequest
- offerId
- viewerEmailHash
- devicePublicKey
- attestationChain
- challengeResponse
- viewerSigningKeyId
- viewerSignature
```

Request dikirim kembali ke owner melalui QR, teks, file, atau mailbox Drive.

Tahap C — Owner menerbitkan kapsul

Owner:

- 1\ Memvalidasi request.
- 2\ Membentuk ViewProjection.
- 3\ Membuat random contentKey AES-256.
- 4\ Mengenkripsi projection dengan AES-GCM.
- 5\ Membungkus contentKey untuk public key perangkat viewer.
- 6\ Menandatangani seluruh kapsul.

Konsekuensinya, kode owner → viewer → selesai dalam satu tahap tidak lagi memungkinkan. Ada harga UX yang harus dibayar untuk mendapatkan device binding.

4. Gunakan Android hardware single-use key

Android Keystore biasa hanya membuat key sulit diekstrak. Untuk kasus ini, gunakan limited-use authorization:

```
if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S) {
    KeyGenParameterSpec.Builder(
        alias,
        KeyProperties.PURPOSE_WRAP_KEY
    )
    .setMaxUsageCount(1)
```

```

        .setUserAuthenticationRequired(true)

        .setUnlockedDeviceRequired(true)
    }

```

Pemeriksaan kapabilitas:

```

val supportsHardwareSingleUse =

    packageManager.hasSystemFeature(

        PackageManager.FEATURE_KEYSTORE_SINGLE_USE_KEY

    ) || packageManager.hasSystemFeature(

        PackageManager.FEATURE_KEYSTORE_LIMITED_USE_KEY

    )

```

Feature flags tersebut secara khusus menunjukkan kemampuan Keystore untuk menegakkan usage count dalam hardware. StrongBox dapat diprioritaskan, tetapi StrongBox dan hardware-enforced usage count tetap harus diperiksa sebagai kemampuan terpisah. StrongBox memberi isolasi yang lebih kuat daripada TEE pada perangkat yang mendukungnya.

Tier perangkat

```

STRICT

Android 12+

Hardware limited/single-use key tersedia

Hardware-backed secure import tersedia

Kapsul boleh disebut "sekali buka"

SUPPORTED_BEST_EFFORT

Android 12+, tetapi usage count tidak hardware-enforced

Fitur tersedia, tetapi jangan disebut guarantee

UNSUPPORTED

Android lama atau Keystore tidak memenuhi persyaratan

One-Time Viewer dinonaktifkan

```

Jangan diam-diam menurunkan strict mode menjadi software flag. Beri pesan seperti:

> Perangkat ini tidak mendukung proteksi hardware yang diperlukan untuk Mode Sekali Buka.

5. Gunakan WrappedKeyEntry untuk implementasi terkuat

Versi sederhana adalah owner mengenkripsi contentKey dengan public key viewer, lalu app viewer memperoleh contentKey plaintext sebentar di memorinya.

Versi yang lebih kuat adalah menggunakan secure wrapped-key import:

```

Owner:

contentKey

↓ wrap dengan public key perangkat

SecureKeyWrapper ASN.1

Viewer:

WrappedKeyEntry

↓ import langsung

Android Keystore AES key

```

Android mendukung secure import sejak API 28. Pada implementasi Keymaster yang mendukungnya, pembukaan wrapped key dilakukan di secure hardware sehingga key material tidak perlu muncul sebagai plaintext di memori proses aplikasi.

Authorization untuk key yang diimpor sebaiknya dibatasi:

```

algorithm = AES

purpose = DECRYPT

blockMode = GCM

padding = NONE

maxUsageCount = 1

unlockedDeviceRequired = true

userAuthenticationRequired = true

```

Dengan demikian:

- wrapping key hanya dipakai untuk kapsul perangkat tersebut;
- content key non-exportable;
- content key hanya dapat menyelesaikan satu operasi dekripsi;
- key Team tidak pernah masuk perangkat viewer.

Karena secure key wrapper cukup kompleks, gunakan implementasi ASN.1 yang diuji terhadap Android CTS, bukan format buatan sendiri. Google menyediakan contoh impor wrapped key dalam pengujian CTS.

6. Jaminannya adalah at-most-once, bukan exactly-once

Ini penting untuk failure handling.

Misalnya urutannya:

```

AES-GCM doFinal berhasil

→ usage count habis

→ app crash

```

→ UI belum sempat tampil

Tidak mungkin membuat operasi hardware decryption dan rendering Android menjadi satu transaksi atomik. Maka sistem harus memilih:

- At-most-once: tidak pernah memberi kesempatan kedua, tetapi kadang data dapat hilang sebelum terlihat.
- At-least-once: dapat retry, tetapi memungkinkan dilihat lebih dari sekali.

Untuk single-use, pilih at-most-once.

Teks konfirmasi sebelum membuka:

> Kapsul hanya dapat dibuka satu kali. Jika aplikasi ditutup, perangkat mati, atau terjadi gangguan setelah proses pembukaan dimulai, akses tidak dapat dipulihkan.

7. State machine yang direkomendasikan

OFFER_RECEIVED

|

▼

DEVICE_BOUND

|

▼

CAPSULE_DOWNLOADED

|

▼

ARMED

| user confirms + authenticates

▼

CONSUMING

| AES-GCM doFinal successful

▼

CONSUMED

| render in memory

▼

SESSION_CLOSED

Aturan penting:

CONSUMED → ARMED

tidak pernah diperbolehkan.

Namun local state bukan security boundary. Record seperti:

```
capsuleState = CONSUMED
```

hanya untuk UX. Security boundary sebenarnya adalah:

- 1\. Viewer tidak memiliki teamKey.
- 2\. Content key terikat ke perangkat.
- 3\. Content key hardware-enforced hanya dapat dipakai sekali.
- 4\. Tidak ada plaintext persisten.

Gunakan alias deterministik:

```
kron/one-time-view/{capsuleId}
```

Jangan izinkan kapsul yang sama diimpor ke alias baru oleh kode aplikasi.

8. Jangan masukkan plaintext ke SQLite

Alur viewer harus benar-benar terpisah dari repository utama:

```
Ciphertext
  ↓ verify owner signature
  ↓ biometric/device credential
  ↓ one cryptographic decrypt operation
ByteArray plaintext
  ↓ validate schema
EphemeralViewModel
  ↓ render
SecureViewerActivity
```

Tidak boleh melewati:

```
Room
SQLite
repository.applySnapshot()
DriveSyncRuntime
SavedStateHandle
DataStore plaintext
```

```
disk cache
temporary JSON file
```

Buat proses terpisah:

```
<activity
    android:name=".viewer.SecureViewerActivity"
    android:process=":secure_viewer"
    android:excludeFromRecents="true"
    android:exported="false" />
```

Keuntungannya:

- komponen viewer tidak perlu membuka database utama;
- lifetime memory lebih mudah diakhiri;
- saat activity ditutup, seluruh secure viewer process dapat dimatikan;
- risiko objek plaintext tertahan oleh singleton atau repository utama berkurang.

Ini masih bukan proteksi terhadap root atau instrumentation, tetapi memperkecil permukaan kesalahan implementasi.

9. Satu sesi foreground

Definisi sesi yang paling jelas:

> Setelah kapsul dibuka, viewer boleh menelusuri data selama layar viewer tetap aktif. Begitu viewer keluar, app masuk background, perangkat dikunci, atau proses mati, sesi berakhir permanen.

Lifecycle:

```
onStop()    → close session
screen locked → close session
process death → session lost
explicit close → close session
reboot      → session lost
```

Jangan menyimpan decrypted model dalam SavedStateHandle. Rotasi layar sebaiknya tidak membuat serialization plaintext. Kamu dapat mempertahankan state hanya dalam memori proses, atau mengunci orientasi khusus viewer jika konsekuensi UX-nya diterima.

Zeroization JVM harus dianggap best effort karena String, Compose text, dan salinan internal runtime tidak selalu dapat dihapus secara deterministik. Tetap lakukan:

```
plaintext.fill(0)
contentBytes.fill(0)
viewModel.clearSensitiveState()
```

tetapi jangan menjadikannya dasar klaim keamanan.

10. Rendering dan AES-GCM

Kapsul sebaiknya dienkripsi sebagai satu AEAD message dan didekripsi dalam satu operasi final.

Jangan menampilkan plaintext sebelum tag GCM tervalidasi. Alur ideal:

```
val plaintext = cipher.doFinal(ciphertext)

validatePayload(plaintext)

render(plaintext)
```

Jika payload dibagi menjadi 100 ciphertext chunk dan setiap chunk membutuhkan operasi Keystore terpisah, `maxUsageCount=1` tidak akan cocok.

Karena itu:

- batasi ukuran kapsul;
 - gunakan projection minimal;
 - satu kapsul = satu ciphertext;
 - lakukan parse setelah autentikasi GCM berhasil.
-

11. Penyimpanan kapsul

Ciphertext boleh disimpan sementara karena tanpa hardware key yang masih usable ia tidak berguna.

Simpan di:

```
context.noBackupFilesDir/one-time-capsules/
```

Android Auto Backup dapat mencakup database aplikasi, sedangkan `noBackupFilesDir` dikecualikan. Tetap tambahkan backup rules eksplisit agar kesalahan konfigurasi tidak membawa metadata sensitif ke backup.

Setelah konsumsi:

- 1\ Delete content-key alias
- 2\ Delete wrapping-key alias
- 3\ Delete ciphertext
- 4\ Delete parsed model references
- 5\ Mark UX state consumed
- 6\ Terminate secure-viewer process saat sesi selesai

Penghapusan file sendiri bukan jaminan utama. Jaminan utamanya adalah ciphertext tidak lagi memiliki key yang usable.

12. Jangan beri akses folder Drive

Dalam desain baru, viewer tidak perlu menjadi member Team Drive.

Pilihan distribusi:

- A. Capsule dikirim sebagai file melalui external channel
- B. Share hanya satu capsule file di Drive
- C. Viewer mengunduh capsule melalui invitation-specific file

Jangan share folder snapshot Team.

Owner dapat menghapus permission Drive untuk mencegah download berikutnya. Namun pencabutan permission hanya mengendalikan akses masa depan; file yang sudah diunduh tetap ada. Google Drive juga memperingatkan bahwa permission langsung pada child item dapat tetap berlaku meski permission parent dihapus.

Pada arsitektur capsule, ini tidak terlalu bermasalah karena salinan ciphertext yang sama tetap terikat pada key perangkat yang sudah habis.

13. Screenshot dan screen recording

Gunakan:

```

window.setFlags(
    WindowManager.LayoutParams.FLAG_SECURE,
    WindowManager.LayoutParams.FLAG_SECURE
)

```

FLAG_SECURE mencegah konten tampil pada screenshot sistem dan non-secure display seperti screen casting pada kondisi yang didukung Android.

Tambahkan juga:

- tidak ada text selection;
- tidak ada copy;
- tidak ada export/share;
- sembunyikan recent-task preview;
- nonaktifkan autofill pada field sensitif;
- jangan mencetak nilai ke log atau crash report;
- jangan mengirim analytics yang mengandung payload.

Tetapi kamera eksternal dan perangkat yang OS-nya sudah dikompromikan tetap tidak dapat dicegah.

14. Expiry dan remote revocation

Expiry offline

expiresAt tidak dapat menjadi jaminan keras bila hanya memakai jam perangkat.

Ia masih berguna sebagai:

- UX rule;
- perlindungan casual;
- validasi saat online;
- batas issuance.

Tetapi jangan menyebutnya tamper-proof.

Revocation sebelum kapsul dibuka

Terdapat dua keadaan:

Kapsul belum diunduh

Owner dapat menghapus file atau permission Drive. Revocation efektif.

Kapsul sudah diunduh

Tanpa pemeriksaan online atau remote authority yang memegang bagian key, owner tidak dapat menjamin revocation.

Jadi kamu harus memilih:

Offline-openable

→ Bisa dibuka tanpa internet

→ Tidak revocable setelah download

Revocable-before-open

→ Harus online saat open

→ Masih membutuhkan authority yang tidak dapat dilewati client

Drive biasa tidak menyediakan primitive “download key tepat satu kali lalu musnah secara atomik”. Karena itu jangan mencoba membuat single-use menggunakan flag atau penghapusan file Drive saja.

15. Threat model yang dapat diklaim

| Skenario | Hasil desain capsule |

| ---|---|

| Tutup lalu buka ulang | Ditolak |

| Duplikasi file kapsul | Tidak memberikan dekripsi tambahan pada instalasi normal |

| Offline setelah download | Dapat dibuka sekali |

| Owner mencabut Drive sebelum download | Akses dapat dicegah |

| Owner mencabut setelah download | Tidak dapat dijamin |

| Backup–restore biasa | Kapsul tidak memiliki key yang dapat dipulihkan |

| Reinstall aplikasi | Device key aplikasi hilang; kapsul tidak dapat dibuka |

| Copy ciphertext ke perangkat lain | Tidak dapat dibuka karena device-bound |

- | Manipulasi SQLite flag | Tidak relevan; plaintext tidak masuk SQLite |
 - | Root/instrumentation/hooksing | Tidak dijamin |
 - | Foto menggunakan perangkat lain | Tidak dapat dicegah |
 - | Pengguna menyalin manual nilai yang dilihat | Tidak dapat dicegah |
-

16. Migrasi dari kode saat ini

Struktur yang saya sarankan:

```

sharing/

|-- team/

|   |-- TeamInvitationCodec
|   |-- TeamJoinPreflight
|   |-- TeamKeyStore
|   +-- DriveSyncRuntime
|
+-- onetime/

    |-- OneTimeViewOfferCodec
    |-- DeviceBindingKeyManager
    |-- ViewCapsuleIssuer
    |-- ViewCapsuleCodec
    |-- WrappedContentKeyImporter
    |-- SingleUseCapabilityChecker
    |-- ViewCapsuleStore
    |-- EphemeralViewSession
    +-- SecureViewerActivity

```

Perubahan paling penting:

- role = VIEWER
- open invitation envelope
- obtain teamKey
- store teamKey
- join Team
- sync snapshot into SQLite

```

\+ accessMode = ONE_TIME_CAPSULE

\+ generate device-bound wrapping key

\+ send device request to owner

\+ receive signed encrypted projection

\+ import one-use content key

\+ decrypt once in secure viewer process

\+ never join Team

\+ never store teamKey

\+ never apply snapshot to SQLite

```

Nama fitur yang lebih jujur

Aku menyarankan nama teknis internal:

```
AtMostOnceViewCapsule
```

Nama UI:

```
Sekali Buka
```

Deskripsi UI:

> Data hanya tersedia dalam satu sesi pada perangkat ini. Setelah layar ditutup atau sesi terputus, data tidak dapat dibuka kembali.

Hindari klaim:

> Mustahil disalin atau direkam.

Penilaian akhirnya

Arsitektur sekarang: tidak dapat memberikan single-use guarantee.

Arsitektur capsule tanpa server:

- kuat terhadap pembukaan ulang normal;
- kuat terhadap sync ulang;
- kuat terhadap salinan file ke perangkat lain;
- dapat hardware-enforced pada perangkat API 31+ yang mendukung;
- tidak memerlukan penyimpanan teamKey;
- tidak memerlukan SQLite plaintext;
- tetap tidak aman terhadap perangkat yang sudah dikontrol attacker atau penyalinan visual.

Jadi fitur ini layak dibuat, asalkan kontraknya disebut at-most-once cryptographic opening, bukan jaminan bahwa informasi hanya pernah diketahui satu kali.